

SGA

- jedinec = binární řetězec
- parental selekce = ruletová
- křížení = jednobodové (důležitější než mutace)
- mutace = bit-flip (malá $p_n \rightarrow$ průměrně 1 bit v jedinci)
- environmental selection : $P(t+1)$ nahradí $P(t)$

VĚTA O SCHÉMATECH

schéma = řetězec 0, 1, *

př. schéma $0*1*$ kóduje jedince 0010, 0011, 0110, 0111

0, 1 = pevné pozice

order $o(S) = \#$ pevných pozic

defining length $d(S) =$ vzdálenost 1. a poslední pevné pozice

fitness $F(S)$ v populaci = $\text{avg} \{F(x) \mid x \in S, x \in P\}$

$\hookrightarrow$! závisí na populaci

V: Schémata s malou definující délkou, malým řádem a nadprůměrným fitness se při běhu GA exponenciálně množí

- malá $d(S) \rightarrow$ malá šance, že ho zničí křížení
- malý $o(S) \rightarrow$ malá šance, že ho zničí mutace
- nadprůměrná $F(S) \rightarrow$ velká šance výběru při selekci

dk. $C(S, t) = \#$ jedinců v $P(t)$ odpovídajících S
 $\rightarrow$ odhadem $C(S, t+1)$

1) selekce

$$p_s(s) = \frac{F(s)}{F(t)} \quad \begin{array}{l} \nearrow \text{fitness populace} \\ \searrow \text{velikost populace} \end{array}$$
$$\rightarrow C(s, t+1) = C(s, t) \cdot n \cdot p_s(s)$$
$$= C(s, t) \cdot n \cdot \frac{F(s)}{F(t)}$$
$$= C(s, t) \cdot \frac{F(s)}{F_{\text{avg}}(t)} \quad \nearrow \text{průměrná } F \text{ populace}$$

↓ nadprůměrná fitness:

$$F(s, t) = F_{\text{avg}}(t) (1 + e) \quad \nearrow e \% \text{ nadprůměr}$$

↓

$$C(s, t+1) = C(s, t) \cdot (1 + e) \quad \nearrow \text{exponenciální růst}$$

$$C(s, t+1) = C(s, 0) \cdot (1 + e)^t$$

2) křížení

$$p_d(s) = \frac{d(s)}{m-1} \quad \rightarrow P \text{ zničení schematu}$$

m = délka jedince, $m-1$ = # pozic pro křížení

malá $d(s)$ → malá $p_d(s)$

$$p_s(s) = 1 - \frac{d(s)}{m-1}$$

p_c = pravděpodobnost křížení

$$\hookrightarrow C(s, t+1) \geq \underbrace{C(s, t) \cdot (1 + e)^t}_{\text{selekce}} \cdot \left(1 - p_c \cdot \frac{d(s)}{m-1}\right)$$

3) mutace

p_m = P mutace → bit nepřezíží

$$p_s(s) = (1 - p_m)^{o(s)}$$

→ malá p_m → $p_s(s) \approx 1 - p_m \cdot o(s)$

$$\hookrightarrow C(s, t+1) \geq C(s, t) \cdot (1 + e)^t \cdot \left(1 - p_c \cdot \frac{d(s)}{m-1} - p_m \cdot o(s)\right)$$

↓

pro malá $o(s)$ a $d(s)$ dostaneme exp. růst

Building blocks hypothesis

GA hledá optimální řešení problému kombinací schemat s malými $d(s)$ a $o(s)$ a nadprůměrnou fitness (building blocks jsou ta schemata)

$C(s, t)$ je expected value $\rightarrow$ funguje dobře na nekonečné populaci
 $\rightarrow$ v praxi nefunguje na malých populacích

př. $(111*****)$ a $(*****11)$ mají nadprůměrnou $F(s)$
ale $F(111*****11) \ll F(000****00)$

rozptyl fitness $\rightarrow$ velké rozdíly $F(s)$ v různých populacích

EXPLORATION VS. EXPLOITATION

exploration $\rightarrow$ prohledávání nových oblastí, nevyžívám znalosti
 $\rightarrow$ není moc efektivní

exploitation $\rightarrow$ využití znalostí
 $\rightarrow$ nejspíš skončím v lokálním optimu

↓

GA adaptivně vyvažuje $\rightarrow$ hledá rovnováhu

1-armed bandit

$\rightarrow$ obyčejný automat s 1 páčkou, s nějakou pravděpodobností dostanu reward

2-armed bandit

- 4 arm má jinou expected value a varianci
- máme omezený počet penízků
- jak identifikovat a využít arm, který lépe platí?

→ snažíme se odhadnout statistické hodnoty

řešení: lepší arm hraje exp. vícrát, ale 2. pořadí občas hraje
(naš odhad nemusí být správný)

k-armed bandit

- reinforcement learning, optimalizace

↓

GA alokuje více pokusů pro úspěšnější schemata,
ale stále nechává prostor pro méně úspěšné

problem: SGA nesampluje schemata nezávisle

REPRESENTACE A OPERATORY

selekce

- ruletová
- turnajová
- stochastic universal sampling → jen 1 pozice na roulette wheel
→ ostatní = shifts o stupeň $\frac{1}{n}$

celočíslné kódování

unbiased mutace = nová náhodná hodnota
biased mutace = nová hodnota je random shift od původní
např. stará + random číslo z normálního rozdělení

n-bodové křížení

uniformní křížení = u všem 50/50 z kterého rodiče

floating point kódování

$$0 < a < 1$$

aritmetické křížení → průměr nebo konv. komb. $o = ap_1 + (1-a)p_2$

EVOLUCE KOOPERACE

- v přírodě i společnosti je spousta kooperace
- ale Darwinismus je inherently competitive
- ↳ jak se tedy altruismus mohl vyvinout?

teorie:

- evoluce na skupinách jedinců → co jedinci, kteří nepomáhají?
- zachování téměř identických genů in close relatives
→ altruismus cizinců nebo jiných druhů?
- selfish gene → jednotkou evoluce je gen, nikoliv jedinec
- reciprocal altruism → vzájemné benefity
(paralely s opakovaným věznovým dilematem)

Věznovo dilema

	C ^{cooperation}	D ^{deception}		C	D
C	-1/-1	-3/0	C	R/R	S/T
D	0/-3	-2/-2	D	T/S	P/P

temptation > reward > penalty > suckers payoff

vzájemná kooperace je lepší než vzájemná deception

ale $T > R$ a $P > S$

takže dominantní strategie je deception

strategie je **dominantní** pro agenta i, pokud dává lepší nebo stejné výsledky jako $\forall$ jiná proti $\forall$ strategii agenta j

strategie s_i a s_j jsou v **Nashově equilibriu**, když

- při s_i má s_j nejlepší možný výsledek
 - při s_j má s_i nejlepší možný výsledek
- ↳ **pure strategies**

mixed strategies = random selection z pure strategies

Nashova věta: V hra s konečným # strategií má v mixed strategiích Nashovo equilibrium

řešení je **Pareto optimální**, pokud není jiná strategie, která by zlepšila výsledek agenta bez zhoršení výsledku jiného hráče

→ mutual deception je Nashovo equilibrium a jediné Pareto optimální řešení
přestože spolupráce maximalizuje společný výsledek

iterované věžňovo dilema

$$T > R > P > S, \quad 2R > T + S$$

→ nevyplatí se střídát C a D

pro pevně daný # her je nejlepší pokaždě deceive

tit-for-tat → začít s kooperací a pak kopírovat kroky oponenta

4 důležité vlastnosti úspěšných strategií

- 1) niceness → don't deceive first
- 2) provocability → quickly punish deception
- 3) forgiveness → quickly calm down
- 4) clarity → simple, so others understand you

EVOLUČNÍ STRATEGIE

- optimalizace reálných fci s mnoha parametry
- floating point čísla (vektory)
- jedinec = [geny, endogenní parametry]

↳ ovlivňují evoluci

→ meta evoluce = evoluujeme i samotné parametry evoluce

nejlepších

$(M+L)$ ES → vybírám M jedinců z M starých a L nových

(M,L) ES → vybírám M jedinců z L nových ($M < L$)

1 potomek z libovolného # rodičů

vyběr rodičů čistě náhodný → nezávisí na fitness

mutace → nejlepší je, když mutace vytvoří lepšího jedince
ve 20% případech ($\frac{1}{5}$ rule)

↳ lze dynamicky podle úspěšnosti mutace měnit
std dev normálního rozdělení (σ)

· větší σ = exploration, menší σ = exploitation

nejjednodušší ES:

minimize $f: \mathbb{R}^n \rightarrow \mathbb{R}$

jedinec = vektor $(x_1, \dots, x_n)$

populace s $n=1$

1 potomek

mutace = změna složky vektoru z normálního rozdělení

rozptyl měním → balance exploration / exploitation

(tedy bez crossoveru)

křížení = rekombinace

↳ víc rodičů → 1 jedinec

< uniformní
aritmické (průměry)

endogenní parametry mutují dříve než jedinec

DIFERENCIÁLNÍ EVOLUCE

• upravená ES

mutace: pro x_i zvolím náhodně x_a, x_b, x_c

$v_i = x_a + F \cdot (x_b - x_c)$, $F \in \langle 0, 2 \rangle$ ^{difference vektor} mutační parametr

křížení: $u_{j,i} = v_{j,i}$ s $p = C$, parametr C = pravdep. změny

$u_{j,i} = x_{j,i}$ s $p = 1 - C$

+ 1 pozice j náhodně vybrána, která bude jistě z v_i

selektce: pokud $f(u_i) \leq f(x_i) : x_i \leftarrow u_i$

↑
minimalizují f

↳ $i \in 1, \dots, N$

↑
rand/1, rand/2: $v = x_a + F(x_b - x_c + x_d - x_e)$ ^{2 rozdílové vektory}

best/1: $x_a = x_{best}$

PARTICLE SWARM OPTIMIZATION

• žádné klasické křížení ani mutace

• jedinec = floating point vektor

p_b = pozice s nejlepší fitness jedince

g_b = nejlepší p_b mezi všemi particles (jedinci)

pro 1 částici:

$$v \leftarrow v + c_1 \cdot \text{rand} \cdot (p_B - \text{aktuální pozice}) + c_2 \cdot \text{rand} \cdot (g_B - \text{aktuální p.})$$

$$\text{aktuální} \leftarrow \text{aktuální} + v$$

(c_1, c_2 learning rate konstanty, typicky $c_1 = c_2 = 2$)

s GA společně: náhodný init, fitness, stochastické update metody
rozdíl: částice mají paměť, bez gen. operatorů, info je z lepších k horším

MICHIGAN VS. PITTSBURGH

= základní evoluční přístupy pro strojové učení

Michigan

jedinec = 1 rozhodovací pravidlo

levá strana: featurea $\rightarrow$ ^{true false don't care} 1/0/*

prava strana: co dál (klasifikace)

populace = systém pravidel (kompletní klasifikační systém)

evoluce nemusí být generační

LCS (learning classifier system)

- prava část obsahuje messages

- levá část má receptor na messages

$\rightarrow$ bufferují zprávy a potřebují distribuovat reward mezi

$\downarrow$ celý řetězec pravidel

Bucket brigade algorithms

- 1 jedinec má váhu (strength) $\rightarrow$ v podstatě fitness

části: prostředí, populace pravidel, mechanismus selekce akce,
mechanismus distribuce odměny, GA

ZCS (zero classifier system)

- zjednodušení LCS (bez messages a BB)
- rules = bitmaps
 - it (input) then (outputs)
- pokud pro situaci není pravidlo, je náhodně vygenerováno
 - ↳ COVER operator
- akci z aplikovaných pravidel vyberu ruletou
 - ($\forall$ pravidlo má strength = váhu)
- historii řeším jen na několik posledních kroků

XCS (extended classifier system)

- vylepšení ZCS
- ZCS většinou nevytvoří kompletní systém (pro $\forall$ případ)
- pravidla na začátku řetězce většinou nejsou odměněna a nepřežívají

pravidlo = pětice (c, a, p, e, f)

c = condition

a = action

p = payoff prediction

e = prediction error

f = fitness použita v GA

↳ založena na accuracy $k = h \cdot \left(\frac{e}{e_0}\right)^{-r}$

↳ h, e_0, r parametry

GA běží na předchozím action setu

Pittsburgh

- jedinec = množina pravidel (kompletní systémy)
- komplikované operátory i evaluace

GIL

- binární klasifikace
- jedinec má implicitní klasifikační skupinu
- jedinec = disjunkce komplexů
- komplex = konjunkce selektorů (z 1 proměnné)
- reprezentace bitmapou
- operátory pro jedince, komplexy i selektory

TSP

- permutační reprezentace
→ seznam měst v pořadí

Crossover :

- PMX** (partially mapped crossover) → zachování pozic
- 2 bodové → půlka zůstane, zbytek doplním

1 2 3 | 4 5 6 7 | 8 9



1 8 2 4 5 6 7 9 3

4 5 2 | 1 8 7 6 | 9 3



4 2 3 1 8 7 6 5 9

- OX** (order crossover) → zachování relativního pořadí

1 2 3 | 4 5 6 7 | 8 9



4 5 6 7

9 3 ~~1~~ ~~2~~ ~~18~~ ~~7~~

2 1 8 4 5 6 7 9 3

4 5 2 | 1 8 7 6 | 9 3



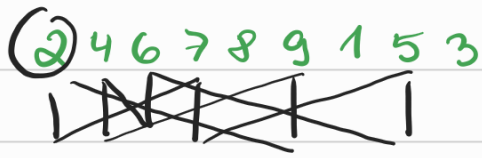
1 8 7 6

~~4~~ ~~5~~ ~~2~~ ~~1~~ ~~8~~ ~~7~~ ~~6~~ ~~9~~ ~~3~~

3 4 5 1 8 7 6 9 2

CX (cyclic crossover) → zachováva absolutní pozice

v cestě



2 4 6 7 8 9 1 5 3

4 6 2 8 7 1 9 3 5

ER (edge recombination) → zachování hran

→ list hran

→ preferuji město s méně hranami

→ většinou lze vytvořit cyklus (selže v 1% příp.)
popř. preference hran, které existují 2x

inicializace:

- nearest neighbors
- edge insertion

mutace:

- prohození měst
- prohození podcest
- inverze
- posunutí podcesty
- prohazování hran na několika městech

GENETICKÉ PROGRAMOVÁNÍ

• evoluce programů

jedinec: program reprezentovaný jako syntaktický strom

terminály = proměnné, konstanty

neterminály = operace

křížení: prohození podstromů

mutace: podstrom $\leftarrow$ náhodný podstrom

selekcce často turnajová

př. aritmetické, bodové výrazy